

Progettazione di Applicazioni Mobili

Flutter Programming

Widgets – Layout

Stefano Burigat

Dipartimento di Scienze Matematiche, Informatiche e
Fisiche - Università di Udine

stefano.burigat@uniud.it



Introduction

- Flutter has well over 100 widgets available out-of-the-box, each with its own options, methods, and events
- The next slides provide information on the subset of those widgets that are used more regularly, organized in the following logical groups:
 - Layout
 - Input
 - Navigation
 - Dialogs, Popups and Messages

Layout widgets

- The layout widgets help you organize your user interface and structure your application in various ways
- Layout in Flutter mainly comes down to a grid structure, which means rows and columns

```
import "package:flutter/material.dart";
void main() => runApp(MyApp());
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(title : "Flutter Playground",
    home : Scaffold(
      body : Center(
        child : Row(
          children : [
            Text("Child1"),
            Text("Child2"),
            Text("Child3")
          ]
        ) // Row
      ) // Center
    ) // Scaffold
  ); // MaterialApp
}
```

Layout: MaterialApp

- The MaterialApp widget implements quite a bit of “plumbing” for us, so it’s always a good idea to start with it
 - Note that it doesn’t matter if you’re developing for iOS, you can still use the MaterialApp widget as the top-level widget
- MaterialApp notable properties
 - title: single-line string of text that is used by the device to identify the app to the user
 - color: defines the primary color used for the application in the OS’s interface
 - theme: takes as its value a ThemeData widget and further describes the colors used for the app
 - home: the top of the widget tree for the starting screen of your app (typically, a scaffolding widget)

Layout: Scaffold

- The basic Scaffold widget implements the basic layout structure of a screen in your app and takes care of many common app UI elements:
 - `floatingActionButton`: allows your app to support a Floating Action Button
 - `drawer`: allows your app to have a sliding drawer for hidden functionality
 - `bottomNavigationBar`: allows your app to have a navigation bar at the bottom
 - `backgroundColor`: enables you to define the background color of the page
 - `body`: a child widget

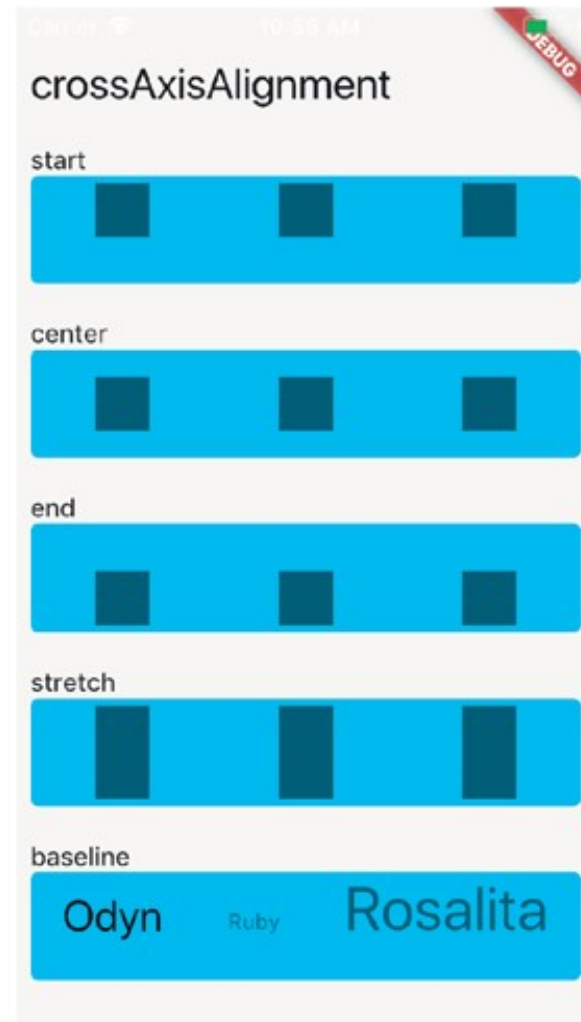
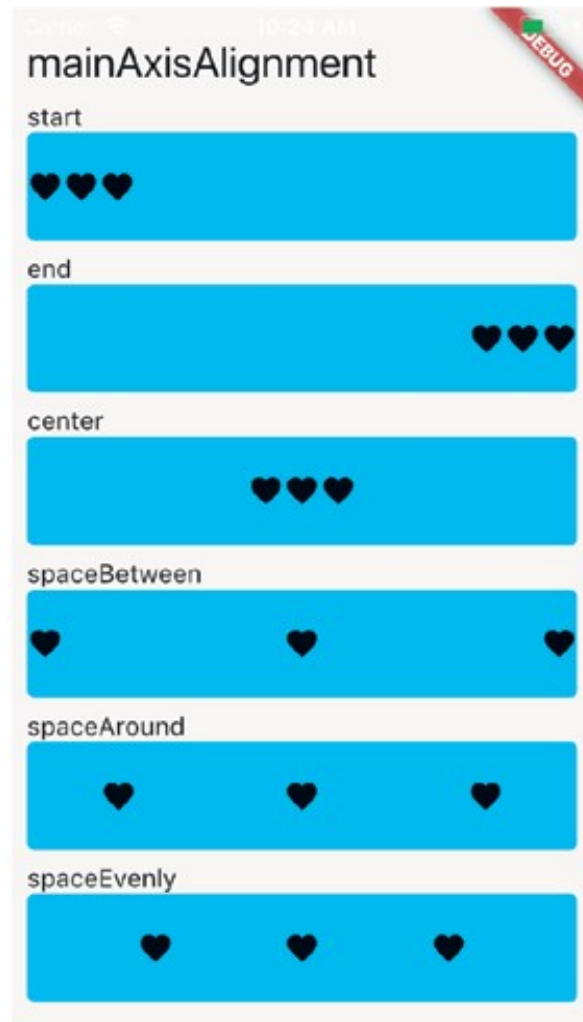
Layout: Center

- The Center widget will center all its children within itself
- By default, a Center widget will fill whatever space its parent widget allows (in this case, the entire screen)
- In the code, the child of the Center is a single Row widget, which means that the Row will be centered within the Center and thus centered on the screen

Layout: Row and Column

- The Row and Column widgets allow you to organize the grid of widgets on the screen
- Each can have one or more children (specified through the children property), and those children will be laid out horizontally (across the screen) in the case of the Row widget and vertically (down the screen) for a Column widget
 - Rows and columns are not scrolling
 - You can nest Row widgets within Column widgets, and vice versa, allowing you to create arbitrarily complex grid structures
 - The mainAxisAlignment property tells the row or column how to position the children along the main axis (e.g., MainAxisAlignment.center tells the Row to center them horizontally)

Layout: Row and Column



Layout: Text

- A Text widget displays a string of text in a single style
- Interesting properties:
 - overflow: tells Flutter what to do when the text overflows the bounds of its container (using `TextOverflow.ellipsis`, for example, causes ... to be appended to the end)
 - style: the style (size, font, color, etc.) to use for the text, in the form of a `TextStyle` widget
 - `textAlign`: lets you determine how text should be aligned horizontally
 - `textScaleFactor`: tells Flutter the number of font pixels for each logical pixel unit and thereby scale the text

Layout: Expanded

- The Expanded widget causes its child to fill all available space

```
Expanded(child : Text("Child2") )
```

- By using expanded and setting its *flex* property, we can give each widget in a Row or Column its own weight

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Test",
      home: Scaffold(
        body: Center(
          child: Row(
            children: [
              Expanded(
                flex: 2,
                child: Text("Child 2"),
              ), // Expanded
            ],
          ), // Row
        ), // Center
      ), // Scaffold
    ); // MaterialApp
  }
}
```

Layout: Align

- The Align widget is similar to the Center widget but has more flexibility
 - This widget aligns its child within itself and can also optionally size itself based on the child's size
 - The key to using it is the alignment property
 - If you set that to `Alignment.center`, then you've just created a Center widget
 - `Alignment.center` is a static instance of `Alignment` which has x and y values of 0 and 0. If you have values of -1 and -1, then that represents the top left of the rectangle, and 1, 1 is the bottom right

Layout: Container

- The Container widget is a jack-of-all-trades that combines what a number of other widgets provide into one sleek package
- If you want to mimic the Center widget you can use

```
Container(alignment : Alignment.center, child...
```

- If you want to put some padding around the second Text widget in the previous example, you can write

```
Container(padding : EdgeInsets.all(20.0), child : Text("Child2") )
```

Layout: Container, Padding, Transform

- If you want to scale the second Text to 200% you can use

```
Container(transform : Matrix4.identity()..scale(2.0), child :  
Text("Child2") )
```

- However, you can do the same operations by using specific widgets such as Padding and Transform

```
Padding(padding : EdgeInsets.all(20.0), child : Text("Child2") )
```

```
Transform.scale(scale : 2, child : Text("Child2") )
```

Layout: ConstrainedBox, FittedBox, RotatedBox, SizedBox

- Flutter offers several “box” components that act a lot like Row and Column and Container, but which provide various positioning, sizing, and other manipulations on its single child
- ConstrainedBox is used to impose additional restraints on its child
 - For example, you want to make the second Text widget in the previous example be forced to a minimum width, you can wrap it in a ConstrainedBox and define that constraint on it

```
ConstrainedBox(constraints : BoxConstraints(minWidth : 200.0), child :  
Text("Child2"))
```

Layout: ConstrainedBox, FittedBox, RotatedBox, SizedBox

- The BoxConstraints class offers some properties for defining a constraint, with minWidth, minHeight, maxWidth, and maxHeight being the most commonly used
- FittedBox scales and positions its child within itself according to a *fit* property

```
ConstrainedBox(constraints : BoxConstraints(minWidth: 200.0), child :  
FittedBox(fit: BoxFit.fill, child : Text("Child2") ) )
```

This scales the Text widget, but in contrast to the previous scale example, this also repositions the Text, so it remains centered, and it scales it up to be a minimum width of 200 pixels in width, so it scales the height to maintain the aspect ratio automatically

Layout: ConstrainedBox, FittedBox, RotatedBox, SizedBox

- The RotatedBox gives us a way to rotate its child

```
RotatedBox(quarterTurns : 3, child : Text("Child2") )
```

The quarterTurns property is the number of clockwise quarter turns to rotate the child by

- The SizedBox widget forces its child to have a specific width and height

```
SizedBox(width : 200, height : 400, child : Text("Child2") )
```

Note that the actual text is by default left aligned and top aligned inside the Text widget

Layout: Divider

- The Divider widget is a straightforward one that displays a one device pixel thick horizontal line, with a little bit of padding on either side
- A Divider can only be horizontal so when the layout is in a Row it won't display
 - Try using a Divider between each pair of Text widgets in the previous example (after having changed Row to Column)

Layout: Card

- A Card widget is a Material design widget that is essentially just a box with rounded corners and a slight drop shadow around content
- Typically, it's used to show some related information in a logical grouping

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Flutter Playground",
      home: Scaffold(
        body: Center(
          child: Card(
            child: Column(mainAxisSize: MainAxisSize.min, children: [
              Text("Child1"),
              Divider(),
              Text("Child2"),
              Divider(),
              Text("Child3")
            ]), // Column
          ), // Card, Center, Scaffold, MaterialApp
    );
  }
}
```

Layout: Drawer

- The Drawer widget is most usually given as the value to the drawer property of a Scaffold widget
- This widget is a Material design panel that slides in horizontally from the left to provide a way for the user to activate app functionality or navigate through the app and which is hidden until called upon
- Another widget, AppBar, goes typically along with a Drawer because it automatically provides an appropriate IconButton to show and hide the Drawer

Layout: Drawer

- The actual contents of the Drawer are entirely up to you, though a ListView is typical, often with the first child being a DrawerHeader widget, which provides a common way to display user account status information
- For iOS, the CupertinoNavigationBar widget is the rough equivalent to the AppBar widget

```
return MaterialApp(  
  title: "Flutter Playground",  
  home: Scaffold(  
    appBar: AppBar(  
      title: Text("Flutter widgets"),  
    ), // AppBar  
    drawer: Drawer(  
      child: Column(  
        children: [  
          Text("Child1"),  
          Divider(),  
          Text("Child2"),  
          Divider(),  
          Text("Child3")  
        ],  
      ), // Column  
    ), // Drawer  
    body: Center(  
      child: Row(children: [  
        Text("Child1"),  
        Text("Child2"),  
        Text("Child3")  
      ])),  
  ])); // Row, Center, Scaffold, MaterialApp
```